

LabReSiD26 – Hands-On 5

Davide Ferrara – 518629

1 Esercizio 1: Analisi di `server_select.c`

Domanda 1 – Differenza tra `server_fd` e `client_fd`

`server_fd` è il socket di ascolto, il suo unico scopo è ricevere le richieste di connessione. Quando `select()` segnala che `server_fd` è pronto, significa che un nuovo client vuole connettersi — si chiama `accept()`.

Il socket di connessione (`client_fd`) viene creato da `accept()` ogni volta che un client si connette. È il canale dedicato a quella specifica connessione: su di esso si usano `recv()` e `send()`. Ne esiste uno per ogni client connesso contemporaneamente.

Domanda 2 – Perché `client_fds[]` inizializzato a `-1` e non `0`

I file descriptor partono da 0: `fd=0` è `stdin`, un fd valido. Inizializzare `client_fds[]` a 0 avrebbe due conseguenze errate:

- Ogni slot risulterebbe già occupato (`client_fds[i] != -1` sarebbe vero), impedendo l’inserimento di qualsiasi client reale, come dimostrato dal seguente output:

```
1 >>> troppi client, rifiuto fd=4
```

- `FD_SET(0, &read_set)` aggiungerebbe `stdin` al set monitorato da `select()`, producendo notifiche spurie ogni volta che l’utente scrive da tastiera.

Con `-1` nessun fd reale assume mai quel valore, quindi ogni slot con `client_fds[i] == -1` è libero.

Domanda 3 – Perché `max_fd` parte da `server_fd` e non da 0

`select(nfds, ...)` monitora i file descriptor da 0 fino a `nfds-1`. Il primo argomento deve quindi essere il fd più grande tra quelli monitorati, più uno. All'avvio l'unico fd presente è `server_fd` (nel nostro caso `fd=3`), quindi `max_fd = server_fd` e la chiamata diventa `select(4, ...)`.

Se si inizializzasse `max_fd = 0`, la chiamata diventerebbe `select(1, ...)`: il kernel scandisce il range `[0, 0]`, ovvero solo `fd=0` (`stdin`), e ignora completamente `server_fd=3`. Il server si bloccherebbe su `select()` indefinitamente anche se un client tenta di connettersi come verificato sperimentalmente, in cui il server non produce alcun output nonostante le connessioni in arrivo.

Domanda 4 – Perché `FD_ZERO` e `FD_SET` ad ogni iterazione

`select()` sovrascrive il set in input lasciando solo i fd pronti in quel ciclo. Per non perdere gli fd non serviti, il set va ricostruito ad ogni iterazione usando `client_fds[]` come rubrica di verità.

Domanda 5 – Disconnessione del secondo client

Il server non crasha. Quando `recv()` ritorna 0 (connessione chiusa dal client), il server chiude il fd con `close()`, imposta lo slot a -1 liberandolo, e ricalcola `max_fd` scorrendo tutta la rubrica. Gli altri client continuano a funzionare normalmente.

Output osservato (tre client connessi, terzo disconnesso con CTRL+C):

```
1 [+] nuovo client connesso fd=4
2 [+] nuovo client connesso fd=5
3 [+] nuovo client connesso fd=6
4 [-] client disconnesso (fd=6)
5 [fd=4] messaggio: "ciao" (4 byte)
6 [fd=5] messaggio: "ciao" (4 byte)
```

Domanda 6 – `select()` può ritornare con `n > 1`

Sì, `select()` può ritornare con `n > 1` quando più fd diventano pronti nello stesso istante. Lo script `test_multifd.sh` 5 lo dimostra: apre 5 connessioni, attende 1 secondo, poi tutti e 5 i client inviano il messaggio contemporaneamente. L'output mostra prima 5 cicli con `n=1` (una connessione alla volta) e poi un ciclo con `n=5` (tutti i messaggi arrivano insieme):

```

1 >>> select() ritorna: 1 fd pronti // client 1 si
   connette
2 >>> select() ritorna: 1 fd pronti // client 2 si
   connette
3 >>> select() ritorna: 1 fd pronti // client 3 si
   connette
4 >>> select() ritorna: 1 fd pronti // client 4 si
   connette
5 >>> select() ritorna: 1 fd pronti // client 5 si
   connette
6 >>> select() ritorna: 5 fd pronti // tutti mandano
   insieme
7 [fd=4] messaggio: "messaggio dal client 1" (22 byte)
8 [fd=5] messaggio: "messaggio dal client 2" (22 byte)
9 [fd=6] messaggio: "messaggio dal client 3" (22 byte)
10 [fd=7] messaggio: "messaggio dal client 4" (22 byte)
11 [fd=8] messaggio: "messaggio dal client 5" (22 byte)

```

Il valore di ritorno di `select()` dipende quindi da quanti fd sono pronti al momento della chiamata, non è sempre 1.

Con `./test_multifd.sh 20` il server smette di accettare connessioni oltre la decima: i client dall'undicesimo in poi vengono rifiutati con il messaggio `[!] troppi client`. Questo non è un problema di `select()`, ma un limite della struttura dati: `MAX_CLIENTS = 10` fissa la capacità massima della rubrica `client_fds[]`. I fd oltre `fd=13` (i primi 10 slot occupano fd 4–13) vengono chiusi immediatamente dopo `accept()`.